Git: how we work

The proposal promises "GitHub, feature-branch strategy, pull-request review". This is how we keep that promise.

The branches

```
main                the finished, working version — only the lead merges into it (from dev)
 └── dev            where finished tasks come together — pull requests go here
     ├── feature/b5-feedback-save      your work, one branch per task
     └── fix/calendar-holiday-icon
```

- You **branch from** `dev` and open your pull request **into** `dev` (the default branch on GitHub).
- The lead reviews and merges it into `dev`. When `dev` is tested and stable, the lead merges `dev` into `main`.
- You never merge into `main` or `dev` yourself.

The rules

1. **`main` and `dev` are protected.** Nobody pushes to them. Everything arrives through a pull request the lead approves.
2. **One task = one branch = one pull request.** Small and focused; never two tasks in one branch.
3. **Small commits, one change each.** "Add the feedback table" is one commit; "the whole of B-5" is not.
4. **Your own name on your own work.** Set it once:

   ```
   git config --global user.name  "Your Name"
   git config --global user.email "the email of your GitHub account"
   ```

5. **Never commit** `.env`, `vendor/`, uploads, logs or a database dump (`.gitignore` stops most of them — check `git status` before every commit anyway).
6. **Never rewrite shared history:** no `git push --force` on `main` or `dev`, no editing a database update that is already in `dev` (make a new one).

Branch names

`type/short-description`, lowercase, words joined with `-`. Put the task number in when there is one.

Type	For	Example
`feature/`	something new	`feature/b5-feedback-save` · `feature/b3-record-achievement`
`fix/`	a bug	`fix/calendar-holiday-icon` · `fix/dropdown-escape-key`
`refactor/`	same behaviour, cleaner code	`refactor/notice-card-variant`
`docs/`	documents only (`team/`)	`docs/checklist-b5`
`chore/`	tools, config, set-up	`chore/check-sh-warning`

Commit messages (Conventional Commits)

```
type(scope): what it does, in the present tense, under 72 characters
```

Example	
`feat(feedback): save a teacher's feedback for a student`	new behaviour
`feat(feedback): add the audience choice (student / parent / both)`	
`fix(calendar): show the holiday icon on the day card`	a bug
`refactor(notice): read the board's words through word()`	cleaner, same behaviour
`db(feedback): add update 041_feedback`	a database update
`test(feedback): note the curl attacks in the checklist`	
`docs(checklist): tick B-5 step 2`	documents
`style(complaints): one property per line`	formatting only

Add a second paragraph (after an empty line) when the *why* isn't obvious.

A whole task, start to finish

Run these inside `code_lecole/` (git works from any folder of the repo; `bin/check.sh` and the paths need it).

```
# 1. Start from the newest dev
git switch dev
git pull

# 2. Make your branch
git switch -c feature/b5-feedback-save

# 3. Work in small steps. After each step:
git status                       # what changed? nothing you didn't mean to?
git diff                         # read it: can you explain every line?
bash bin/check.sh                # 0 errors
git add app/Models/FeedbackModel.php database/updates/041_feedback.sql
git commit -m "db(feedback): add the feedback table"

# 4. Push your branch (the first time with -u)
git push -u origin feature/b5-feedback-save

# 5. Open a pull request into dev on GitHub (or: gh pr create --base dev --fill)
#    Fill in the template. Ask the lead to review.

# 6. The lead asks for changes → fix, commit, push again (the pull request updates by itself)

# 7. After the merge: back to dev, delete your branch
git switch dev
git pull
git branch -d feature/b5-feedback-save
```

Keeping your branch up to date

When `dev` moved on while you were working (someone else's pull request was merged):

```
git switch dev
git pull
git switch feature/b5-feedback-save
git merge dev                    # brings their work into your branch
```

A conflict (`CONFLICT` in the output) means you both changed the same lines:

1. Open the file: Git marks both versions between `<<<<<<<`, `=======` and `>>>>>>>`.
2. Keep the right lines (often both), delete the markers.
3. `bash bin/check.sh`, then `git add <file>` and `git commit`.
4. Not sure? Stop and ask the person who wrote the other side. Never just delete their lines.

The files most likely to clash: `team/CHECKLIST.md`, `routes/web.php`, `config/permissions.php` and `database/updates/` (two people taking the same number — check `dev` for the next free number before you push).

Reviews

- **The author** fills in the pull request template honestly, including "Still open / not checked".
- **The lead** reads every file, runs `bin/check.sh` and the `rules-reviewer`, tries the pages, and either approves or asks for changes with a comment on the exact line.
- **The GitHub check** (`Rule check`) runs `bin/check.sh` on every pull request. Red = not merged.
- Merges use **"Create a merge commit"** (not squash), so every small commit and its author stay in the history.

Useful commands

Command	Does
`git status`	what changed, what is staged
`git diff` / `git diff --staged`	the changes, line by line
`git log --oneline --graph -15`	the recent history
`git restore <file>`	throw away your changes to one file (careful)
`git restore --staged <file>`	un-stage a file (keeps the changes)
`git commit --amend`	fix the last commit's message — **only before pushing**
`git stash` / `git stash pop`	park unfinished work, get it back
`git branch -a`	every branch